

Car Marketplace System

Object Oriented Programming — Assignment 02

Girdhari Lal

FAST NUCES Karachi | Spring 2026

1. Introduction

This report covers Assignment 02 which extends the Car Marketplace system from Assignment 01. The new concepts added are Inheritance, Polymorphism, Abstraction, Operator Overloading, and Friend Functions. All changes are built on top of the original code without breaking anything.

2. Inheritance (08 Marks)

We created two inheritance hierarchies. The first is User as a base class for Seller, Buyer, and Admin because all three share common data like id, name, phone, and isActive. The second is Vehicle as a base class for Car and Bike because both share brand, model, year, mileage, and color. Repeating this data in every class would be bad design.

Base Class	Derived Class	Reason
User	Seller	Seller is a User who posts listings
User	Buyer	Buyer is a User who searches and buys
User	Admin	Admin is a User who manages the platform
Vehicle	Car	Car is a Vehicle with fuelType added
Vehicle	Bike	Bike is a Vehicle with bikeType added

3. Polymorphism (05 Marks)

We used both function overriding and function overloading.

Function overriding: display() is a virtual function overridden in Car, Bike, Seller, Buyer, and Admin. Each class prints different information even though the function name is the same. showReport() is also overridden in all three User subclasses.

Function overloading: In the Listing class, display() is overloaded with a bool parameter. Calling display() shows full details. Calling display(true) shows a compact one-line version. Same function name, different parameters, different behavior.

Runtime polymorphism is shown in main() using Vehicle pointers:

- Vehicle* v1 = &car; v1->display(); prints Car with fuel type
- Vehicle* v2 = &bike; v2->display(); prints Bike with bike type

Same pointer type, different output — the actual object type decides which function runs.

4. Abstraction (10 Marks)

We created 4 abstract classes with pure virtual functions. No object can be made from these directly. Each has its own separate header (.h) file: IDisplayable.h, ISearchable.h, IReportable.h, and Vehicle.h.

Abstract Class	Pure Virtual Functions	Implemented By
IDisplayable	display(), getType()	Listing, Car, Bike
ISearchable	matchesKeyword(), inBudget()	Listing, Car, Bike
IReportable	showReport(), getID()	Seller, Buyer, Admin
Vehicle	display(), getType(), matchesKeyword()	Car, Bike

5. Operator Overloading (07 Marks)

We overloaded 5 operators across 3 classes. The == and + operators are both implemented as required.

Class	Operator	Returns	Reason
Vehicle	==	brand + model match	Check if two cars are the same model
Listing	==	listing ID match	Check if two listings are the same
Listing	<	price comparison	Find the cheaper of two listings
Listing	+	combined price total	Calculate bundle price of two listings
User	==	user ID match	Check if two users are the same

6. Friend Functions (05 Marks)

We used 3 friend functions. A friend function can access private members of a class without being a member itself. We used them when a function needs private data from two different objects at the same time.

Friend Function	Private Data Accessed	Why Friend is Needed
printCheaper(Listing&, Listing&)	price of both Listings	Reads price from two Listing objects directly
compareSellerRatings(Seller&, Seller&)	rating, name of both	Reads private rating from two Sellers at once
canAfford(Buyer&, Listing&)	budget of Buyer	Checks if private budget covers listing price

7. Integration with Assignment 01 (05 Marks)

All features from Assignment 01 still work correctly:

- Seller and Buyer extend User but original functions like saveFavorite(), addListingID(), showFavorites() still work the same.
- Car extends Vehicle but Listing still uses Car the same way through getCar() and all getters.
- Admin still approves and removes listings through Marketplace.
- Static members, const members, composition, aggregation, and arrays of objects from A1 are all still present.
- All 10 original classes are still there. We added 4 abstract classes and the Bike class on top.

8. Files Submitted

The following files are included in the GitHub repository:

- A2_CarMarketplace.cpp — main source file with all classes and main()
- IDisplayable.h — abstract interface for displayable classes
- ISearchable.h — abstract interface for searchable classes
- IReportable.h — abstract interface for reportable user classes
- Vehicle.h — abstract base class for Car and Bike

9. Conclusion

Assignment 02 successfully extends the Car Marketplace system with all required OOP concepts. Inheritance avoids code repetition by putting shared data in User and Vehicle. Polymorphism lets the same function call behave differently per object type through overriding and overloading. Abstraction defines strict interfaces with 4 abstract classes each in its own header file. Operator overloading makes object comparisons and price combinations natural. Friend functions give controlled access to private data across different objects. The program compiles and runs without errors.